



RV College of
Engineering®

Cyber Physical Systems (CPS)

UNIT - II

Dr. Narasimha Swamy S, Ph. D.

Associate Professor, Department CSE (AIML)

RV College of Engineering, Bengaluru-59

Go, Change the World



Content

→ Introduction to Embedded Systems

- Definition
- Desirable Features and General Characteristics
- Embedded Systems VS. General Computing Systems
- Model of an Embedded System
- Classification of Embedded System
- Examples of Embedded Systems

→ ARM Processors/ Controllers

- The ARM Core
- Features of ARM Processors
- ARM Processor families
Cortex A, Cortex R and Cortex M.

→ Programmed I/O

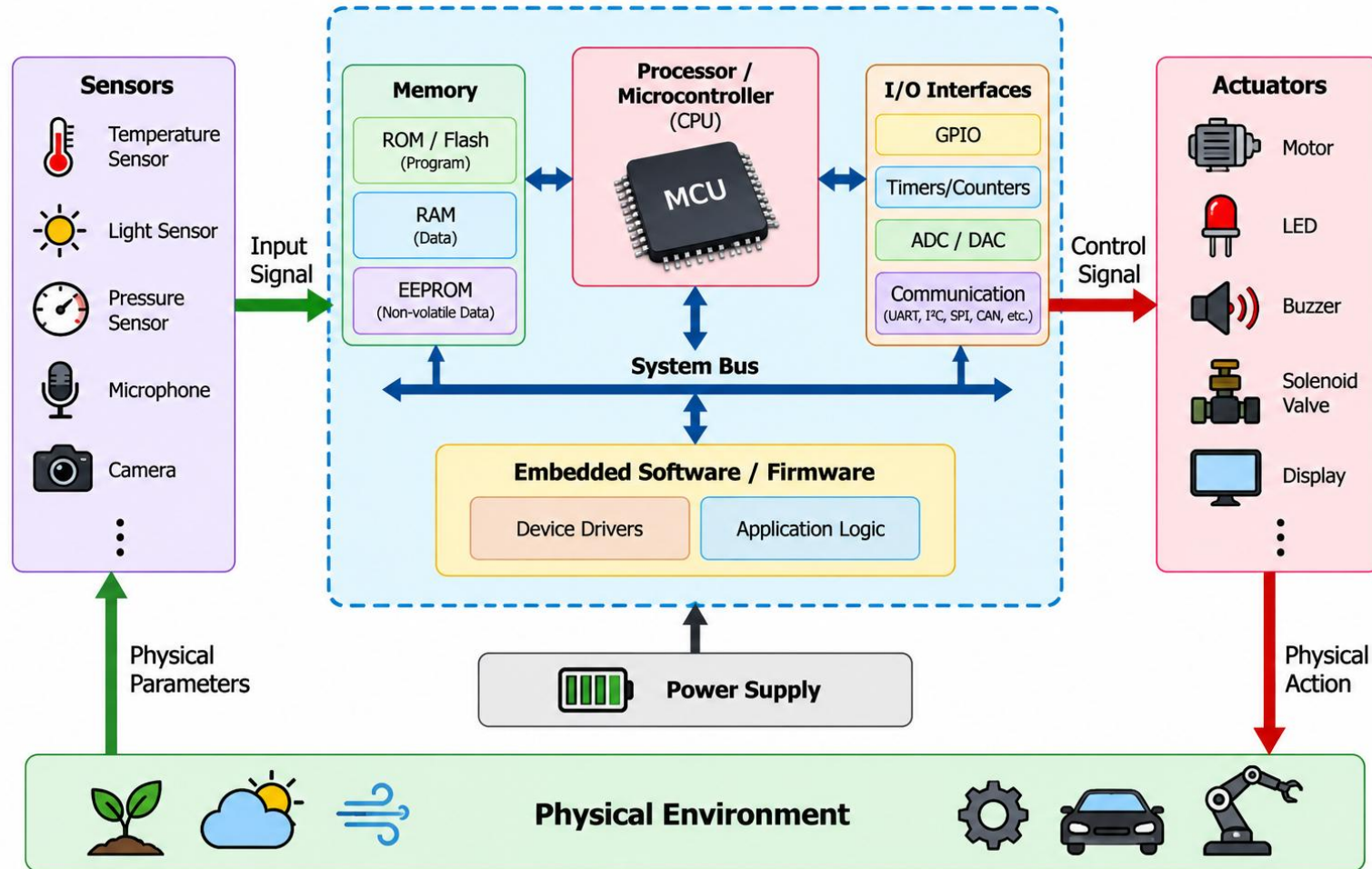
→ Interrupt-driven I/O

→ Parallel and Distributed Computers

- Introduction to parallel architectures
- SIMD computers, MIMD computers

Introduction to Embedded Systems: Definition

An Embedded System is a **computer-based system designed to perform a specific function** (monitor, control, or perform a specific task) or a set of dedicated functions as part of a larger electronic or mechanical system.





Introduction to Embedded Systems: Features and General Characteristics

- It should have **one or a small set of functions** which it is expected to perform efficiently.
- It should be designed for **low-power dissipation**, because many systems are **battery powered**.
- It has **limited memory and limited number of peripherals**.
- Applications are not meant to be **alterable by the user**.
- Many of them are **not accessible directly**, that is, they may be part of the control unit of a larger system, so no interference in operation is possible.
- They need to be highly **reliable**.
- Many of **them need to operate with time constraints**.



Introduction to Embedded Systems: Features and General Characteristics

Now let's try to understand why a PC is not considered to be an embedded system.

- The PC has a large application set, from word processing and computation to communications, printing, scanning and many more.
- Memory is available in various forms: RAM, ROM and secondary memory devices like the hard disk, CDRoms and the like. More memory can be added if the user desires.
- Since the PC is used for various applications, more applications can be added as and when needed.
- The PC can be accessed by input devices like the keyboard, mouse, modem, etc.
- Like any other system, the PC also needs to be reliable, but since it is unlikely to be the part of a very critical system, it can afford to fail once in a while (not a very good idea, though because PCs are used in critical monitoring applications sometimes).
- The applications on the PC need to be fast for better performance, but usually there is no time criticality involved.

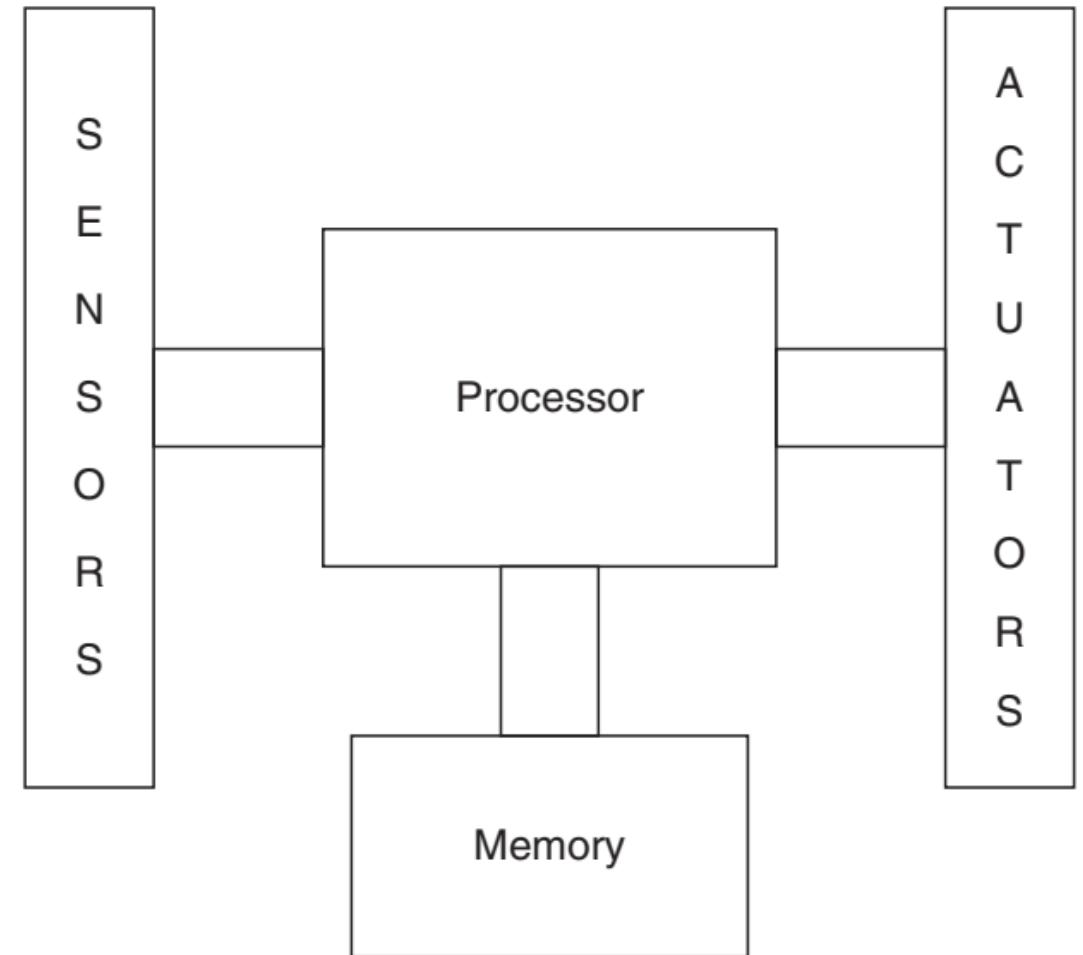


Introduction to Embedded Systems: EC VS. GC

Computer	Embedded Computer
Desktop computers can run a variety of application programs	Embedded computer is normally dedicated to a specific task
Desktop computers generally comprise of Operating system, and allows to install application software's	The embedded computer may or may not have an operating system, and rarely does it provide the user with the ability to arbitrarily install new software
Architecture is complex	Embedded hardware is often much simpler
Desktop computers need the multiple chips	An embedded computer may be implemented in a single chip with just a few support components
Flexibility is high	Lower Flexibility
Higher Computing and Processing power	Less computing and processing power

Introduction to Embedded Systems: Model of an Embedded System

- In its simplest and most general form, an **embedded system consists of a processor, sensors, actuators and memory**
- The idea is that any application should be able to provide solution to a real-world problem, for which some data is to be read in. For this, sensors are needed.
- Data is **processed by the processor and the result of it is given to actuators** which perform appropriate actions.





Introduction to Embedded Systems: Classification of Embedded System

- We can classify **embedded systems on the basis of their complexity**
- Complex operations **imply large amounts of data**, usually. For example, image and video operations need large word lengths and higher clock rates. This needs MCUs also with wide data buses.
- MCUs of **different data and address bus widths are available**. Here **we classify MCUs** on the basis of their data word lengths.
 - **4-bit MCUs:** Some applications deal with very little computation, and in that case 4 bits of data bus could be sufficient. Simple toys and applications which use just switch inputs and directly perform actuation don't need to handle large volumes of data
 - **8-bit MCUs:** The highest volumes of MCUs used are the ones with 8-bit data buses. For moderately complex operations, this is sufficient. The most popular of this is the 8051 family, which was developed by Intel, but which is now manufactured by various other companies as well. Microchip's PIC is another popular family with many different series with varying capabilities. Newer versions of PIC with more and newer peripherals are being developed which makes the PIC series very attractive. Incidentally, the PIC series includes 8-bit, 16-bit and 32-bit MCUs



Introduction to Embedded Systems: Classification of Embedded System

- MCUs of different data and address bus widths are available. Here we classify MCUs on the basis of their data word lengths
- **16-bit MCUs:** There are a few 16-bit MCUs like Intel's 8096, 80196, some versions of PIC, etc. MSP 430 (manufactured by Texas Instruments) is a new 16-bit series, which has very low-power dissipation, and can compete effectively in the new embedded market, which is very particular about power dissipation.
- **32-bit MCUs:** 'ARM' is the most popular 32-bit MCU in use today; it is used in complex applications requiring low power, high speed and good computing capability. The 32-bit MCUs are the ones used in image and video applications and thus find use in the latest mobile phones, iPods, PDAs, etc.



Introduction to Embedded Systems: Classification of Embedded System

Let us discuss some other devices which are also included in the category of embedded systems

1. ASIC: Application Specific Integrated Circuit (IC)

- This is an IC in which complex functional blocks are integrated to make it a complete application. The IC is designed from basics, after defining its application. It can even be that it is tailor-made for a particular customer by the designer company.
- A video codec (coder decoder) is an example of an ASIC. What we get here is a hardware implementation of a complex algorithm, and this hardware implementation will be very efficient and fast (in the case of a sturdy design). ASICs are generally expensive to make, especially because of 'strict' specifications and limited application market.

What is an ASIP?

ASIP stands for 'Application Specific Instruction Set Processor'. It is a processor whose instructions set is tailor-made for a specific application, like graphics, for example. Thus, it will be a sort of trade off design between the programmability features of a CPU and the performance of an ASIC.



Introduction to Embedded Systems: Classification of Embedded System

Let us discuss some other devices which are also included in the category of embedded systems

2. FPGA (Field Programmable Gate Array)

- These devices also are included in the domain of embedded systems. As the name indicates, it is a programmable hardware— a type of hardware which is programmable, that is, reconfigurable even while it is part of a circuit. It is an advanced form of complex programmable logic devices. Here the device density is very high.
- In this, a number of logic cells are interconnected—the logic cells as well as interconnects are programamable using hardware description languages and synthesis tools.
- This makes hardware design cheap and flexible, but the end result is not as efficient or as fast as ASICs. There are a number of well-advanced companies supplying FPGAs—Xilinx, Altera, Altec, etc.



Introduction to Embedded Systems: Classification of Embedded System

Let us discuss some other devices which are also included in the category of embedded systems

DSP Processors

- The above-said processors belong to the set of 'general purpose processors'. They have instruction sets which cater to general arithmetic and logical computations. But for many applications like signal processing where floating point operations and complex arithmetic operations are involved, their performance is unlikely to be sufficiently fast and efficient. Here comes the necessity for processors with instruction sets which are designed for signal processing and complex math operations. They are called DSP processors.
- Where real-time processing of speech, image, video, etc. are involved, they perform superbly. There are many companies manufacturing such DSP processors: Texas Instruments is the leader in the design of DSP processors, and Analog Devices, Nvidia, Lucent, Freescale, etc. also some of them. For many applications, the current trend is to have a general purpose core and a DSP core on the same chip, so that tasks can be partitioned. See Figure 1.7 which is a very popular setup for advanced operations—an ARM core and a DSP core handling different types of computations, along with a number of peripheral controllers—all on the same chip.



Introduction to Embedded Systems: Classification of Embedded System

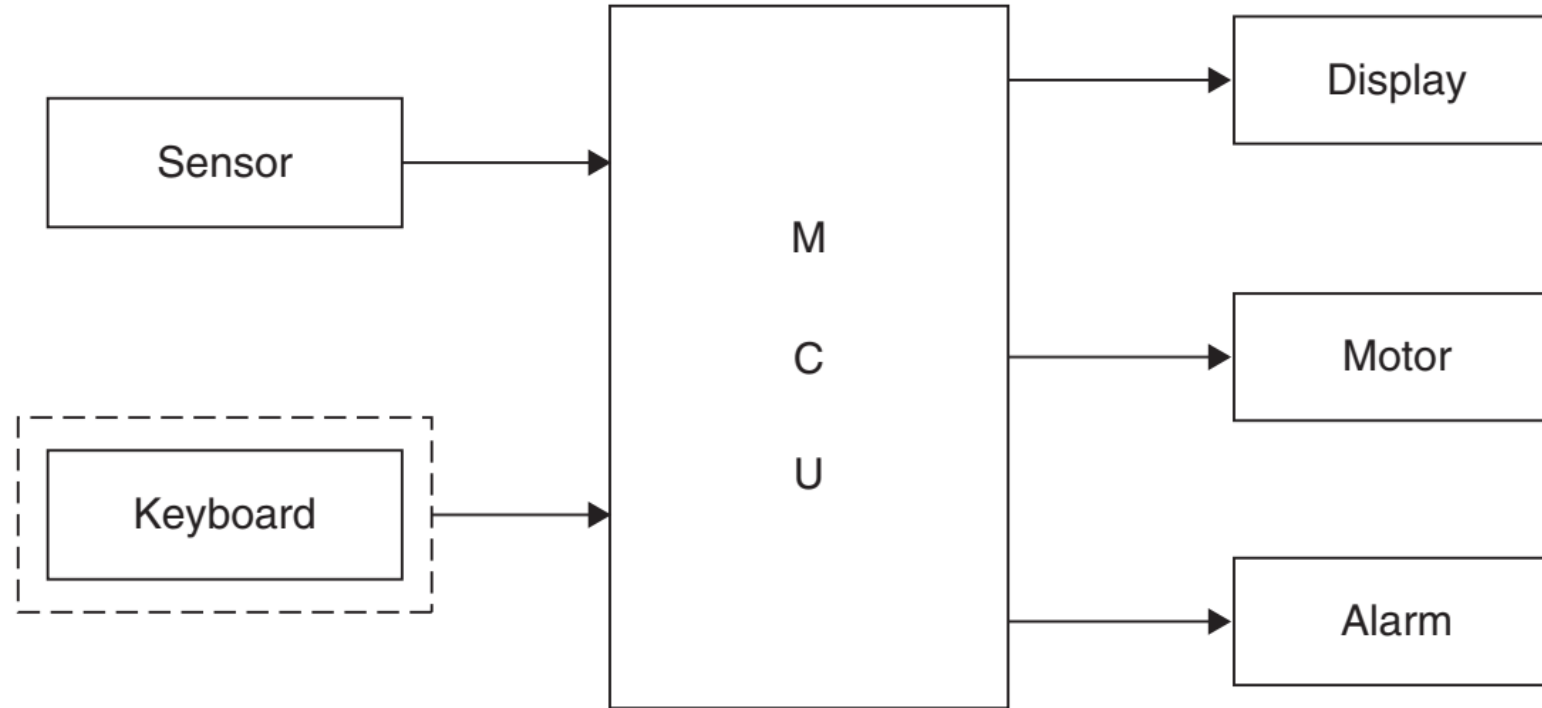
Let us discuss some other devices which are also included in the category of embedded systems

DSP Processors

- The above-said processors belong to the set of 'general purpose processors'. They have instruction sets which cater to general arithmetic and logical computations. But for many applications like signal processing where floating point operations and complex arithmetic operations are involved, their performance is unlikely to be sufficiently fast and efficient. Here comes the necessity for processors with instruction sets which are designed for signal processing and complex math operations. They are called DSP processors.
- Where real-time processing of speech, image, video, etc. are involved, they perform superbly. There are many companies manufacturing such DSP processors: Texas Instruments is the leader in the design of DSP processors, and Analog Devices, Nvidia, Lucent, Freescale, etc. also some of them. For many applications, the current trend is to have a general purpose core and a DSP core on the same chip, so that tasks can be partitioned. See Figure 1.7 which is a very popular setup for advanced operations—an ARM core and a DSP core handling different types of computations, along with a number of peripheral controllers—all on the same chip.

Introduction to Embedded Systems: Examples of Embedded Systems

- Let's examine the embedded system in detail. We will think of a very simple system, as in Figure.



- An MCU is considered the brain of this system, which functions as a temperature monitor/controller. A sensor reads the temperature, and an ADC inside the MCU converts it to digital form. This data is compared to a reference temperature, and if it is above the allowed value, an alarm is activated. Also an output from the MCU is used to start a cooling fan to reduce the temperature. There is a digital display of the temperature as well.



History of ARM

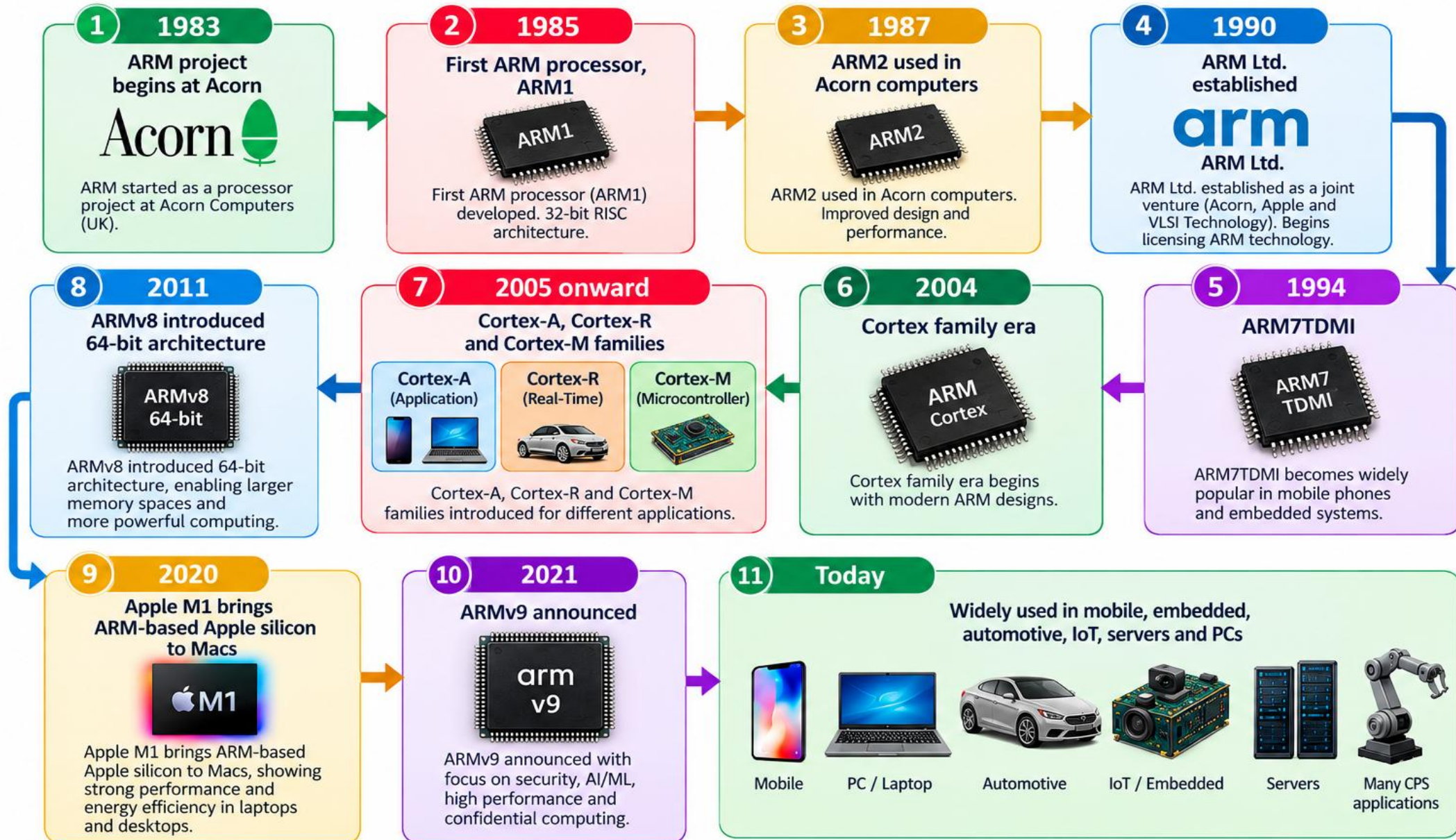
- ARM has a very interesting history because it started as a small processor project for a personal computer and eventually became one of the most widely used CPU architectures in the world.
- Abbreviation
 - Earlier: ARM stands for Advanced RISC Machines
 - Now: ARM stands for Acorn RISC Machine
- ARM originated at Acorn Computers in the United Kingdom. In the early 1980s, Acorn wanted to develop a processor that was: Fast, Simple, Low-cost, and Energy efficient
- Acorn engineers investigated the RISC (Reduced Instruction Set Computer) approach. The basic idea was to use a relatively small set of instructions that can be executed efficiently.
- In 1985, the first ARM processor, ARM1, was developed. It was designed as a 32-bit RISC processor.
- The architecture emphasized: Simple instructions, Large register set, Efficient instruction execution, Low hardware complexity. This became the foundation of the ARM architecture.



History of ARM (Contd.)

- ARM2 Major Early Success: The ARM2 was introduced in the mid-1980s and was used in Acorn's computers.
- One important characteristic was its relatively simple design. Compared with many contemporary processors, ARM could provide good performance while using relatively little hardware.
- This helped establish ARM's reputation for: Performance + Simplicity + Low Power
- In 1990, ARM Ltd. was established as a joint venture involving: Acorn, Apple, and VLSI Technology
- This was an important turning point. Instead of ARM being used only by Acorn, the company began developing ARM technology for licensing to other semiconductor manufacturers. This licensing model became extremely important to ARM's future.
- It became extremely popular in: Mobile phones, Embedded systems, Game devices, and Consumer electronics

History of ARM (Contd.)





ARM Core

- The **core** is the 'processing unit' or the 'computing engine' which has all the computing power, and this aspect is decided by the architecture, which represents the basic design of the processor.
- One **special and unique feature of ARM as a company** is that it designs the core and licenses **this IP** (Intellectual Property) to others.
- This simply means that **the company does not 'fabricate' the chip, but sells only the design.** This design is taken by the licensee, who may or may not add more features (usually peripherals) to the design.
- Sometimes the **buyer can also modify the basic design to a minor extent.** The **buyer company fabricates the design and sells it/uses it for its products**
- There are various ways in which ARM sells its IP.
 - It could be in the form of a soft IP. In this case, the design is sold as RTL (VHDL/Verilog code), and this allows the buyer to modify the design to a certain extent.
 - If the design is sold as a hard IP, it means the buyer gets only the layout or the net list (connection of nets or electronic wires).



The Feature of ARM Processors

Let's discuss the features which have made ARM a very popular processor in the high-end embedded market

1. Data bus width

The processor has a 32-bit data bus width, which means that it can read and write 32 bits in one cycle. For high end applications, having a wide data bus corresponds to a high data bandwidth and is very important. When ARM first made its entry into the field, there were very few embedded processors which had such a wide bus width.

2. Computational Capability

The instruction set of ARM has been cleverly designed to facilitate very good computational capability. Many unique and new methods of fast computation without the necessity of extensive hardware is used. The design of the processor used the RISC approach, but over the years, this philosophy has been diluted to enable the addition of specialized hardware for computationally intensive tasks. In essence, ARM is a RISC processor which has a few CISC features as well



The Feature of ARM Processors (Contd.)

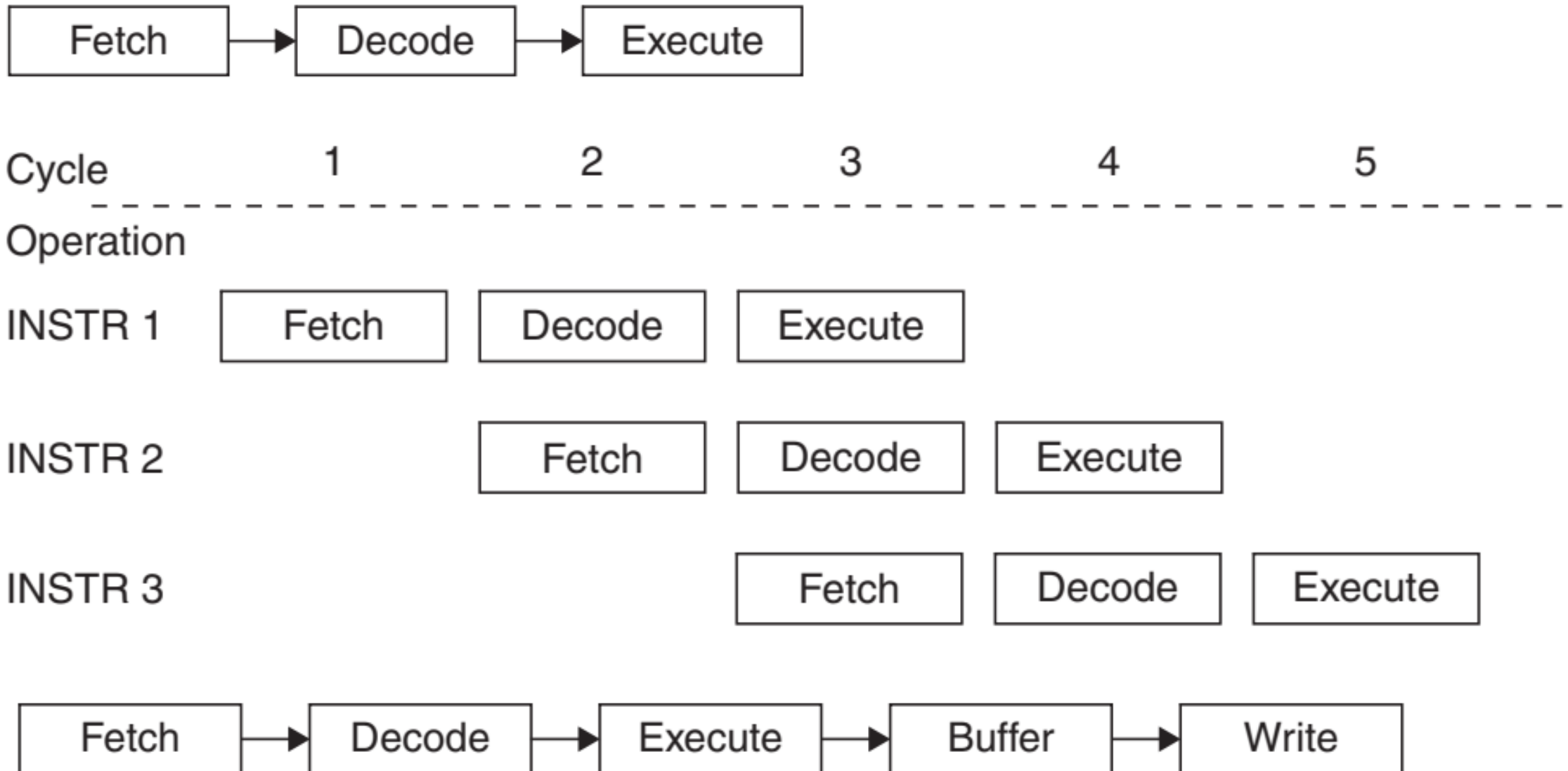
3. Low Power

In the embedded field, power saving is very important, because a large number of devices operate on battery power. Designing lower power processor cores is thus a matter of high priority. How is it that a processor is designed to have low power capability? **Embedded processors operate at low clock frequencies compared to desktop processors. While 3.3GHz is commonly used in the desktop processor field, ARM operates at relatively low frequencies from 60 MHz to at the most 1 GHz.**

4. Pipelining

Pipelining is a fundamental idea in computer architecture, for increasing the speed of operation. The idea is to get many activities to be done in tandem, by dividing the whole instruction processing stage into sub stages. The basic task that any processor does is 'fetch, decode and execute'. In the simplest form of pipelining (3 stage), all the three stages are active all the time. While the first stage is fetching an instruction, the next stage, that is, the decode stage, is busy with the decoding of the previously fetched instruction, and the execute stage is executing the instruction which had been previously decoded.

The Feature of ARM Processors (Contd.)





The Feature of ARM Processors (Contd.)

4. Multiple Register Instructions

Since ARM is a RISC processor, it has instructions which process data which are in registers only – this simply means that data processing instructions do not use of addressing modes in which one operand is in memory. But there are instructions which access memory and load data into multiple registers – also, contents of multiple registers can be stored in memory, with a single instruction.

5. DSP enhancements

Our processor has RISC as its basic policy, but the more advanced members of the family have DSP (Digital Signal Processing) instructions as an enhanced feature. This is where ARM departs from its RISC philosophy but is necessary for surviving in the embedded market. These DSP enhancements are signified by an 'E' in the name as of the ARMv5TE and ARMv5TEJ architectures.



ARM Architecture

- ARM architecture is the design and organization of an ARM processor—how it stores data, executes instructions, handles memory, responds to interrupts, and manages different operating modes.
- **Instruction Set Architecture (ISA)**
 - The term ISA implies the user's i.e.; the programmer's view of the processor, which constitute the instruction set, addressing modes, registers, etc.
 - ISA is the assembly programmer's or compiler designer's view of the processor. We will base most of our discussions on ARM7 which was the first and still the most popular of the ARM processors.
 - Advanced versions may have more enhancements, but the basic architecture is the same.
- **Operating Modes**
 - ARM has seven operating modes which are listed here.
 - Operating mode determines the what the processor is doing currently and what are the resources it uses.
 1. **User:** Unprivileged mode under which most tasks run



ARM Architecture (Contd.)

2. **FIQ (Fast Interrupt Request):** Entered on a high priority (fast) interrupt request
3. **IRQ (Interrupt Request):** Entered on a low priority interrupt request
4. **Supervisor:** Entered on reset and when a software interrupt instruction (SWI) is executed
5. **Abort:** Used to handle memory access violations
6. **Undef:** Used to handle undefined instructions
7. **System:** Privileged mode using the same registers as user mode

■ Register Set

- ARM has 37 registers each of which is 32 bits long. It is the collection of register available in the processor to store data, address, and immediate results.
 1. 1 dedicated program counter (PC)
 2. 1 dedicated current program status register (CPSR)
 3. 5 dedicated saved program status registers (SPSR)
 4. 30 general purpose registers



ARM Architecture (Contd.)

Table 10.4 | Registers in the User Mode

Register Numbers	Designations
R0–R12	General purpose registers
R13	Stack pointer (SP)
R14	Link register (LR)
R15	Program counter (PC)

R0–R12 are general purpose registers, or what may be designated as scratch pad’ registers. These are the registers into which data and address are loaded. They are also ‘the’ registers used in computations.

R13 is the pointer to the stack, and is the stack pointer (SP).

R15 acts as the program counter (PC), which, like in any other processor, is the register which sequences instructions as they are fetched from memory

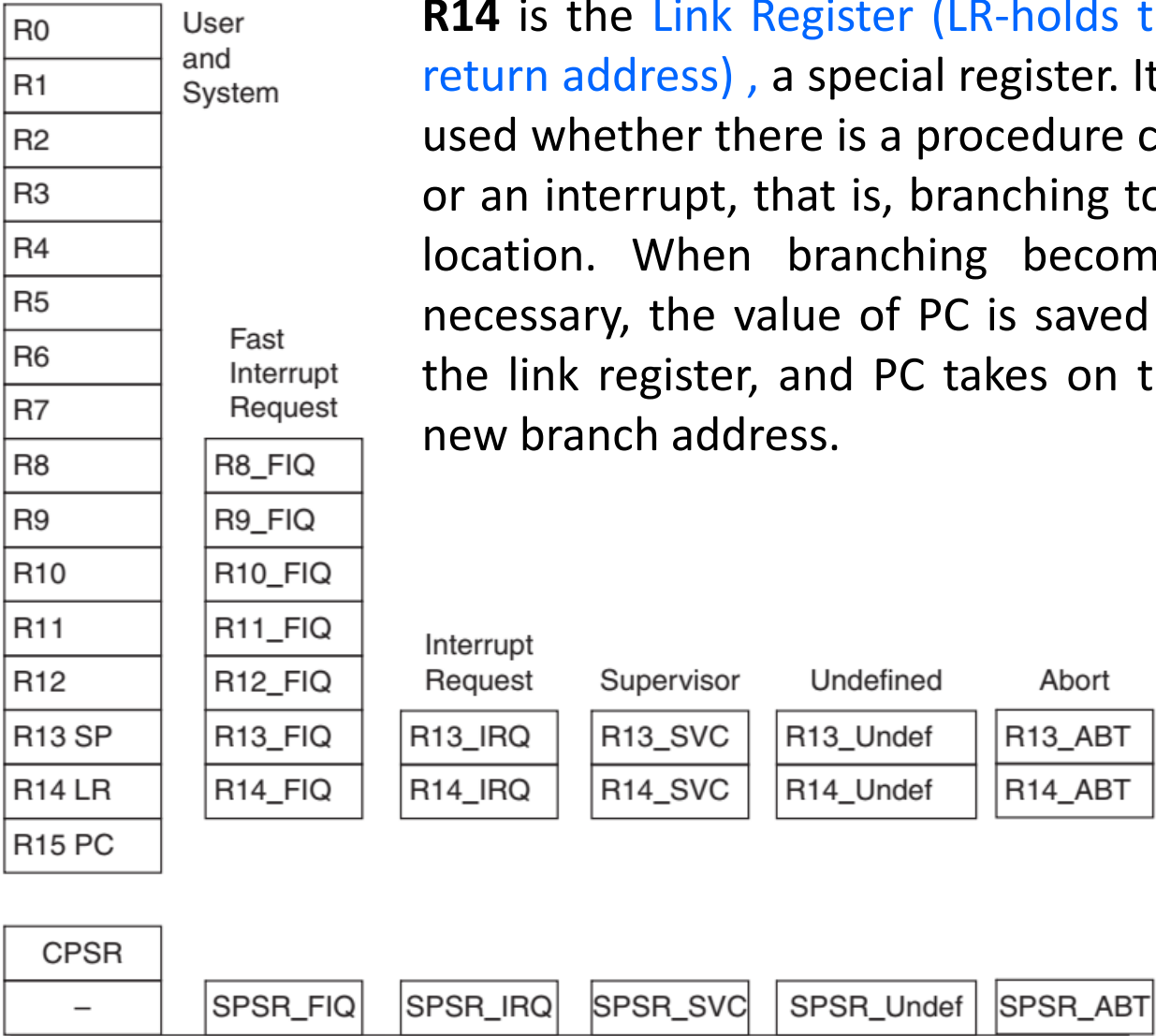


Figure 10.5 | Register set of ARM

R14 is the **Link Register** (LR-holds the return address) , a special register. It is used whether there is a procedure call or an interrupt, that is, branching to a location. When branching becomes necessary, the value of PC is saved in the link register, and PC takes on the new branch address.



ARM Architecture (Contd.)

■ Mode Switching

- We know that there are **seven modes** for the processor, **which implies that it can be switched to different modes**, as decided by the requirement.
- When the processor switches, say, **from the user to another mode**, some of the **user mode registers are replaced by another set of registers**.
- See the **FIQ mode**, for example, in this mode, **R8 to R14 are replaced by another set of registers**, and the **names of these registers are suffixed by FIQ**, like **R14_FIQ, R12_FIQ and so on**.

Why Is it that FIQ uses another set of registers?

- FIQ has a separate/ banked set of registers **to make interrupt response as fast as possible**.
- Normally, when an interrupt occurs, the CPU may need to save the registers currently being used
- FIQ provides **banked registers**: When the processor enters FIQ mode, it automatically switches to: **R8_FIQ, R9_FIQ, R10_FIQ, R11_FIQ, R12_FIQ, R13_FIQ, R14_FIQ**

ARM Architecture (Contd.)

■ Mode Switching

CPSR (Current Program Status Register)

- The **CPSR (Current Program Status Register)** is a very important register, and there is only one such register for the processor. Figure 10.6 and Table 10.5 gives its details.
- The **CPSR contains the information about the current state of the processor**. It has bits which specify the mode, control bits to enable/disable interrupts, and also specifies whether the Thumb or ARM mode is currently in use.

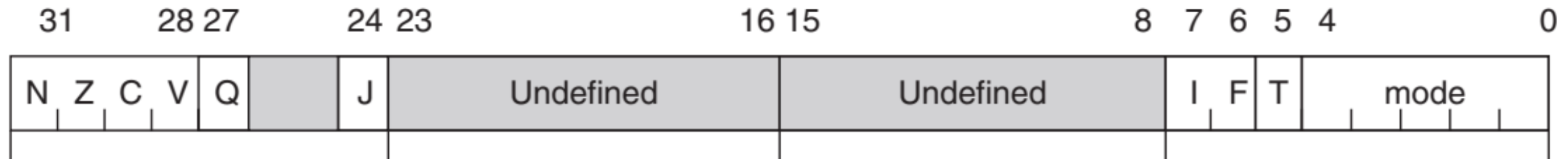


Figure 10.6 | Current Program Status Register (CPSR) bit configuration

ARM Architecture (Contd.)

■ Mode Switching

Bit Nos.	Notation	Interpretation
0 to 4	Mode	Specifies the current mode of operation
5	T	Specifies whether in ARM(T = 1) or Thumb(T = 0) state
6	F	Disables (F = 1)FIQ
7	I	Disables (I = 1)IRQ
8 to 23, 25 to 26	Undefined	
24	J	In Jazelle state (J = 1)
27	Q	Sticky overflow flag
28 to 31	V, C, Z, N	Conditional flags

- Bits 0 to 4 specify the **current mode of operation**. Since there are only 7 modes of operation, only seven mode numbers are valid.
- The **J bit** is for indicating whether the Jazelle state is valid or not.
- The **T bit** specifies whether the current operation is in the ARM or Thumb mode.

- The **contents of this register can be modified only in the highly privileged system mode**. It also contains the condition flag bits. Most of you are likely to know the relevance of the conditional flag bits. But for those who might be new to the concept of flags, here is a concise description.



ARM Architecture (Contd.)

■ Conditional Flags

- **N: Negative Flag** - Status of the MSB of the result of an operation. If we are dealing with signed number $N = 1$ means that the sign bit = 1, which is a negative result.
- **C: Carry Flag** This bit is set if there is an overflow from the MSB of the data being manipulated; this can happen in additions, shifts, rotates etc. It is also set when the result of subtraction is positive. If $R1 - R2$ gives a positive result, $C = 1$, indicates that $R1$ is greater than $R2$. To be precise, let's say that 'A carry occurs if the result of an add, subtract or compare is greater than or equal to 232, or as the result of an inline barrel shifter operation in a move or logical instruction'.
- **Z: Zero Flag** If the result of an arithmetic or logical operation is zero, then $Z = 1$.
- **V: Overflow Flag** This is the overflow flag, which is relevant only for signed operations. It indicates that the sign bit has possibly been corrupted because the result has gone out of the range
- **Q: Sticky Overflow Flag** This flag indicates overflow itself, but it is 'sticky' in the sense that it remains set until explicitly cleared.



ARM Architecture (Contd.)

- Conditional Flags

Saved Program Status Registers (SPSR) There are five 'Saved Program Status Registers', that is, one for each of the 'exception' modes of operation. When an exception, that is, an interrupt occurs, the corresponding SPSR saves the current CPSR value into it (so as to be able to retrieve it on returning to the previous mode). The system mode and user modes do not have SPSRs because they are not entered through the mechanism of interrupts.



ARM Processor Families

- ARM processors are organized into different families/ profiles based on their intended applications

- The most important ARM families are

Cortex-M

Examples: Cortex-M0+, M3, M4, M7, M23, M33, M55, M85.

Cortex-R and

Examples: Cortex-R4, R5, R7, R8, R52.

Cortex-A

Examples: Cortex-A53, A55, A72, A76, A78, A710, A720.



ARM Processor Families: Cortex M

- Cortex-M ARM processors are designed primarily for **microcontrollers and deeply embedded systems**.
- They are optimized for:
 - **Low power consumption**
 - **Low cost**
 - **Fast interrupt response**
 - **Real-time operation**
 - **Small memory footprint**
- Cortex-M based microcontrollers are commonly used in: **CPS, Automotives, Consumer Electronics, Medical Devices, Robotics, Wearables, Drones, Etc.**



ARM Processor Families: Cortex R

- Cortex-R ARM processors are designed for **high-performance, and real-time embedded systems**
- Features
 - **Real-Time & Deterministic:** Fast and predictable response to time-critical events
 - **High Reliability & Safety:** Features such as MPU, error detection, and safety-oriented configurations support dependable operation
 - **High Performance:** More powerful than typical microcontroller-class Cortex-M processors for demanding control applications.
 - **Multicore Support:** Selected Cortex-R processors support multicore real-time processing.
- **Applications:** Automotive ECUs, industrial automation, robotics, storage controllers, and medical systems



ARM Processor Families: Cortex A

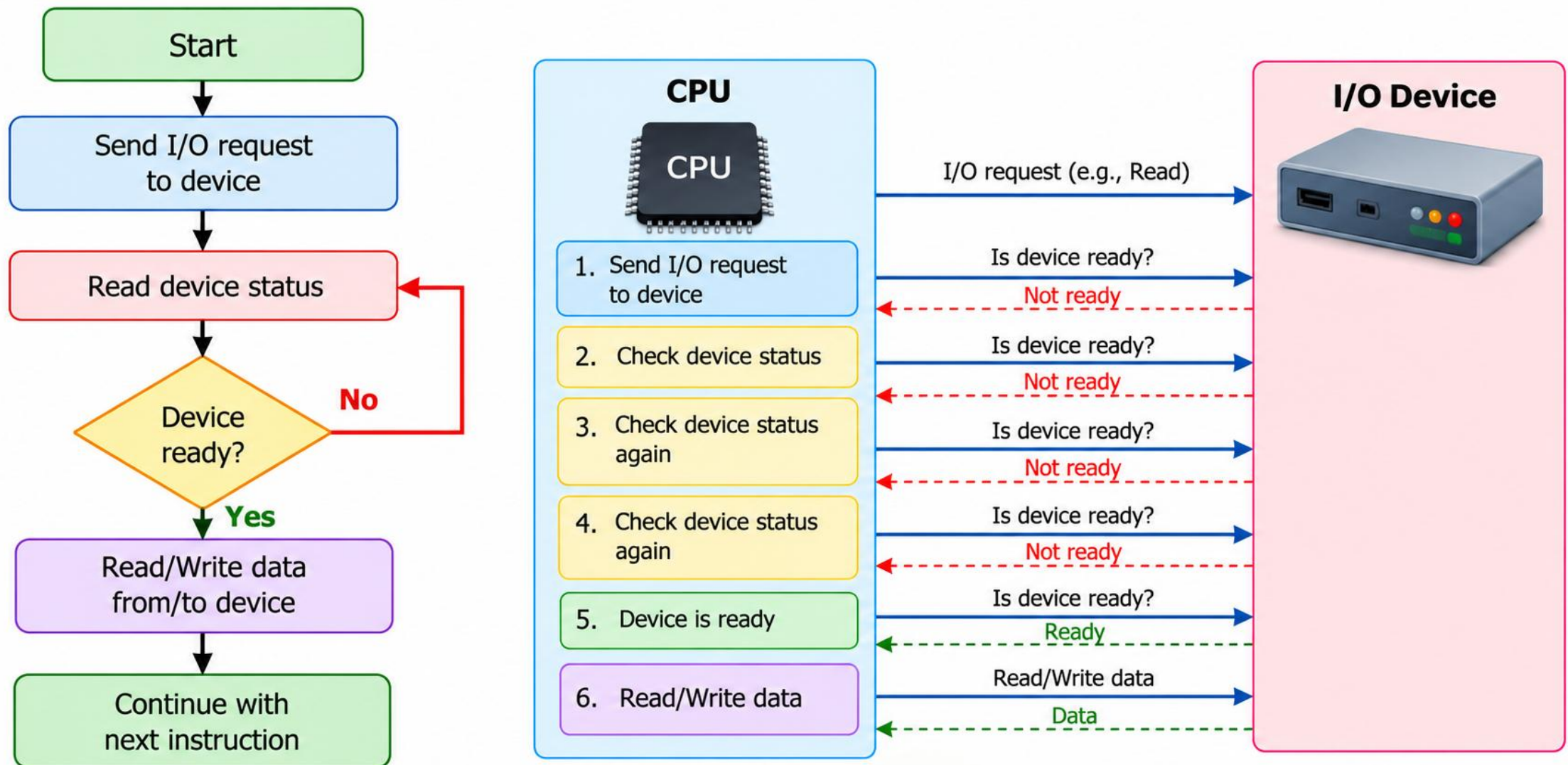
- Cortex-A are the ARM processors designed for high-performance application computing
- Features
 - **High Performance:** Designed for demanding applications, multitasking, and complex computations
 - **Rich Operating Systems:** Supports operating systems such as **Linux, Android, and other full-featured OSs.**
 - **Advanced Computing:** Supports features such as **multicore processing, virtual memory, caches, and floating-point/ vector processing.**
 - **High Connectivity:** Used in systems with advanced networking, graphics, multimedia, and peripheral interfaces.
- **Applications:** Smartphones, tablets, laptops, smart TVs, automotive infotainment, edge computers, and servers



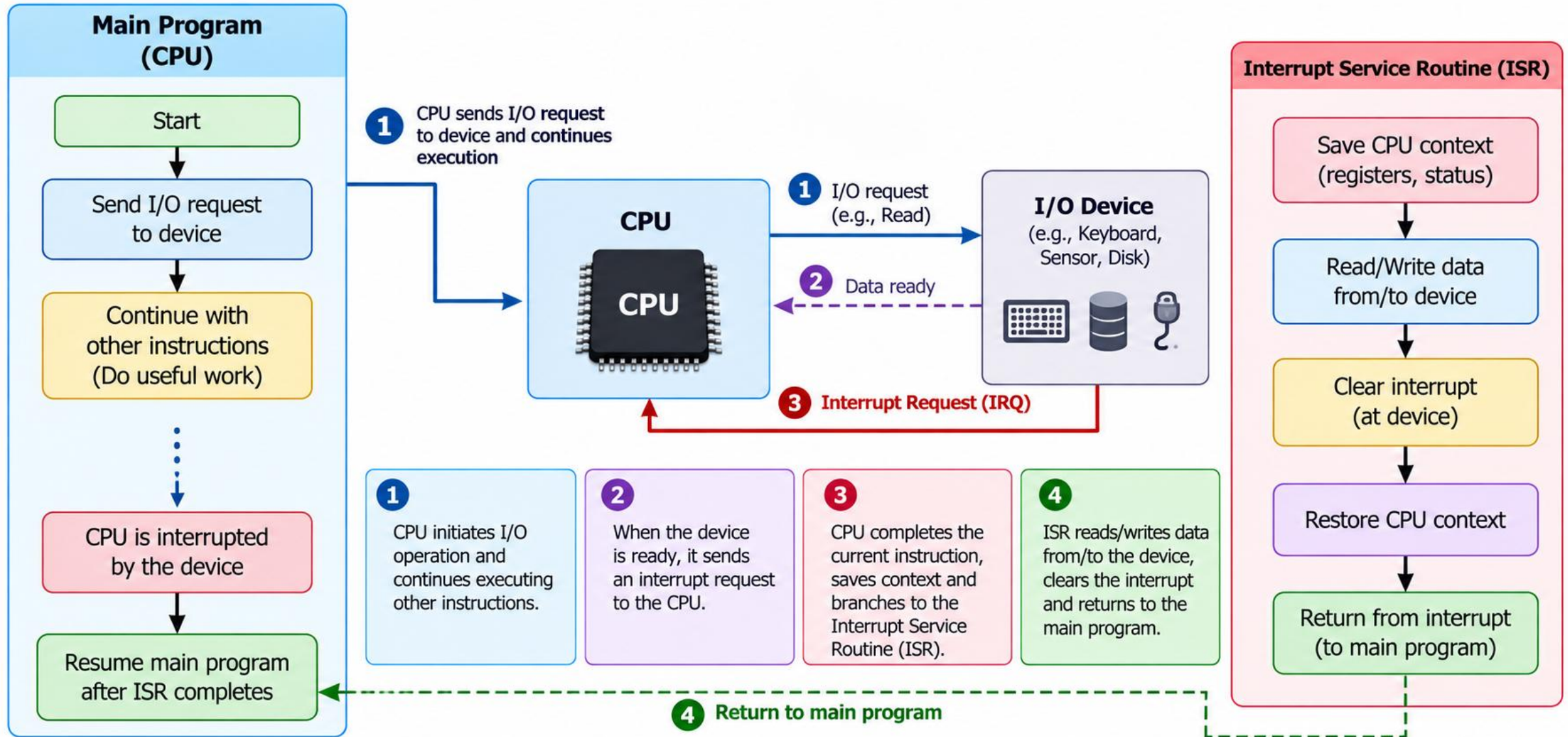
ARM Processor Families: Differences

ARM Family	Primary Purpose	Example Applications
Cortex-M	Microcontrollers	IoT, sensors, embedded systems
Cortex-R	Real-time processing	Automotive, industrial control
Cortex-A	Application processing	Smartphones, laptops, Linux
Cortex-X	High performance	High-end mobile/consumer SoCs

Programmed I/O and Interrupt-driven I/O



Programmed I/O and Interrupt-driven I/O.





Programmed I/O VS. Interrupt-driven I/O.

Feature	Programmed I/O	Interrupt-Driven I/O
CPU checks device?	Yes	No continuous checking
Method	Polling	Interrupt
CPU utilization	Lower efficiency	Better efficiency
CPU waits?	Often yes	Usually no
Device notifies CPU?	No	Yes
ISR required?	No	Yes
Hardware complexity	Simpler	More complex
Suitable for	Simple/ slow systems	Systems with multiple/asynchronous devices



Parallel and Distributed Computers

- Some embedded applications require greater performance than is achievable from a single processor
- It is becoming increasingly common to see embedded systems implemented using parallel processors



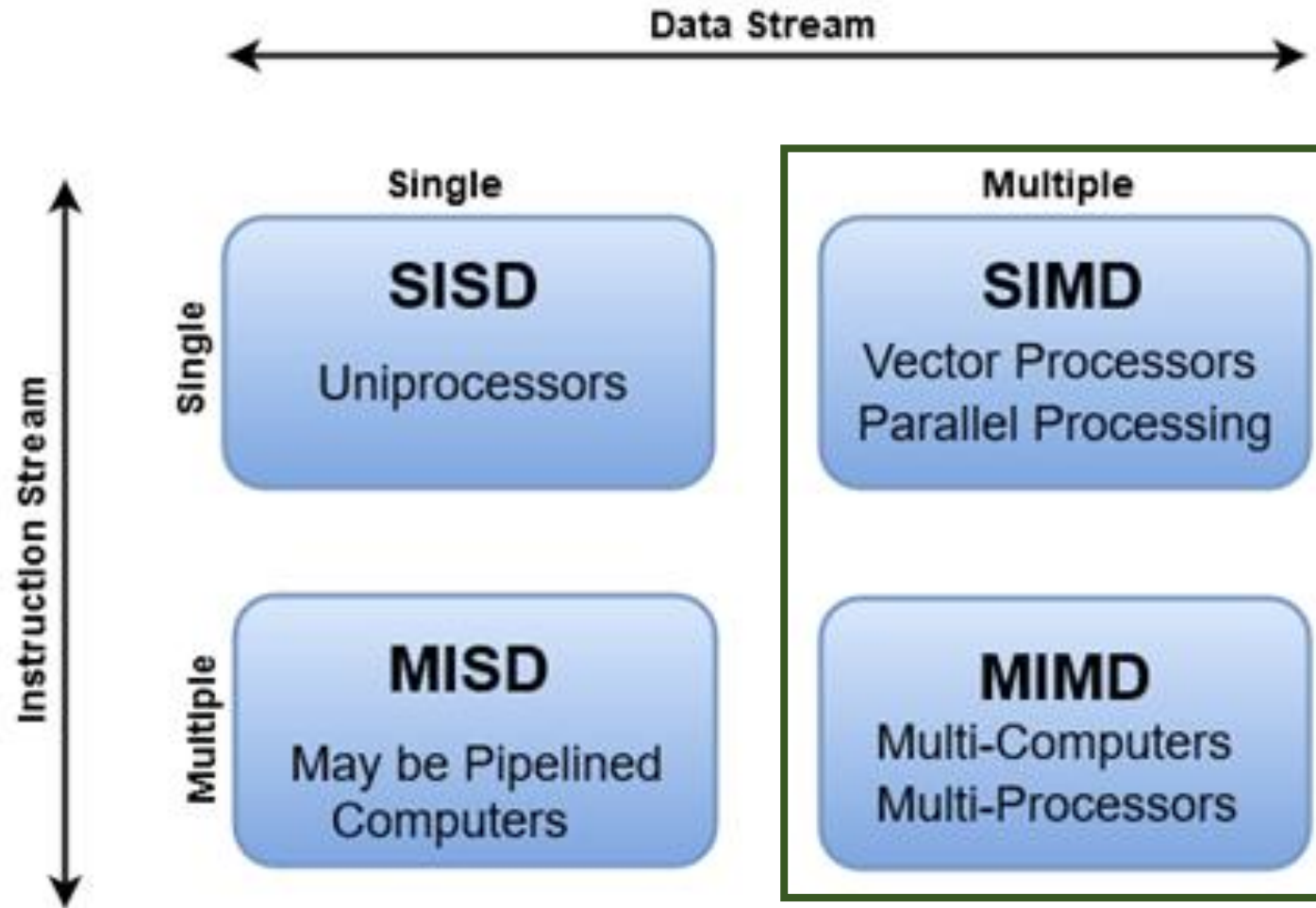
Parallel and Distributed Computers (Contd.)

Introduction to Parallel Architectures

- The traditional architecture for computers follows the conventional, Von Neumann serial architecture
- Computers based on this form usually have a single, sequential processor. The main limitation of this form of computing architecture is that the conventional processor is able to execute only one instruction at a time
- The Algorithms that run on these machines must therefore be expressed as a sequential problem
- A given task must be broken down into a series of sequential steps, each to be executed in order, one at a time.
- Here, we are discussing two parallel Architectures
 - Single Instruction, Multiple Data (SIMD) Architecture
 - Multiple Instructions, Multiple Data (MIMD) Architecture

Parallel and Distributed Computers (Contd.)

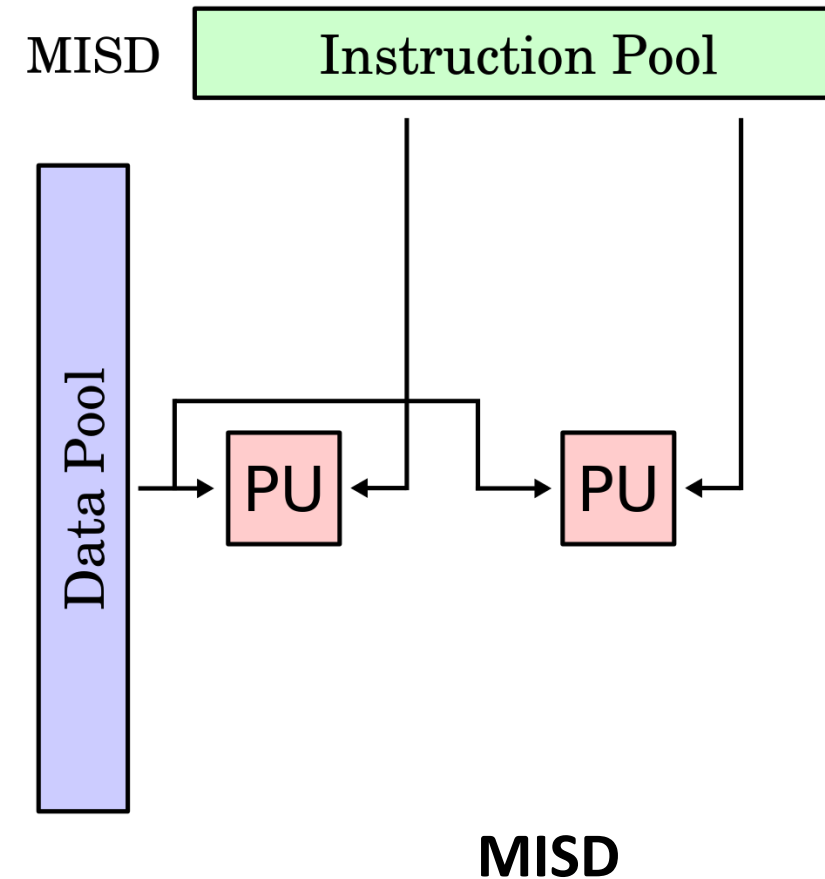
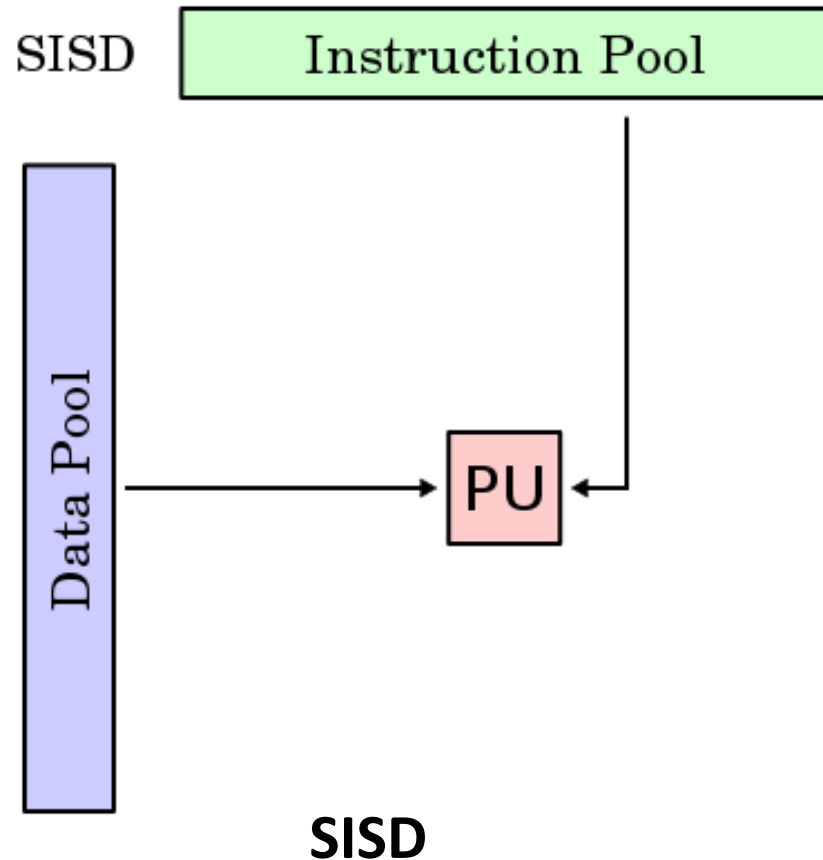
Classification Computer Architecture



Classification of Computer Architecture

Parallel and Distributed Computers (Contd.)

Classification Computer Architecture





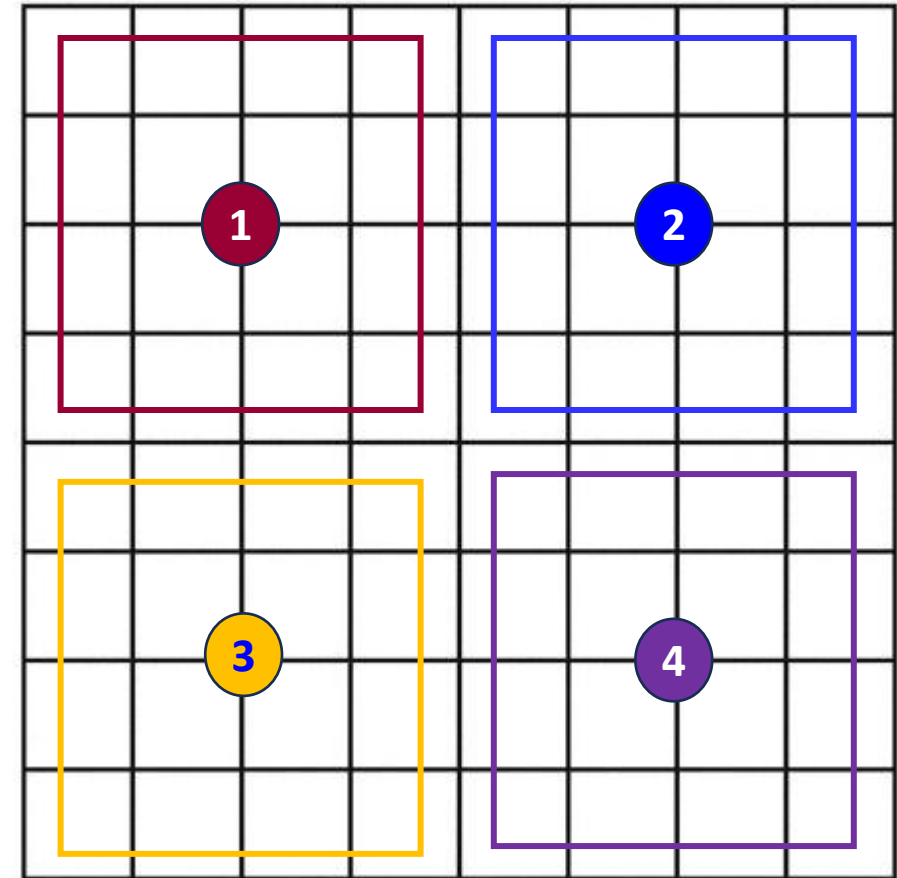
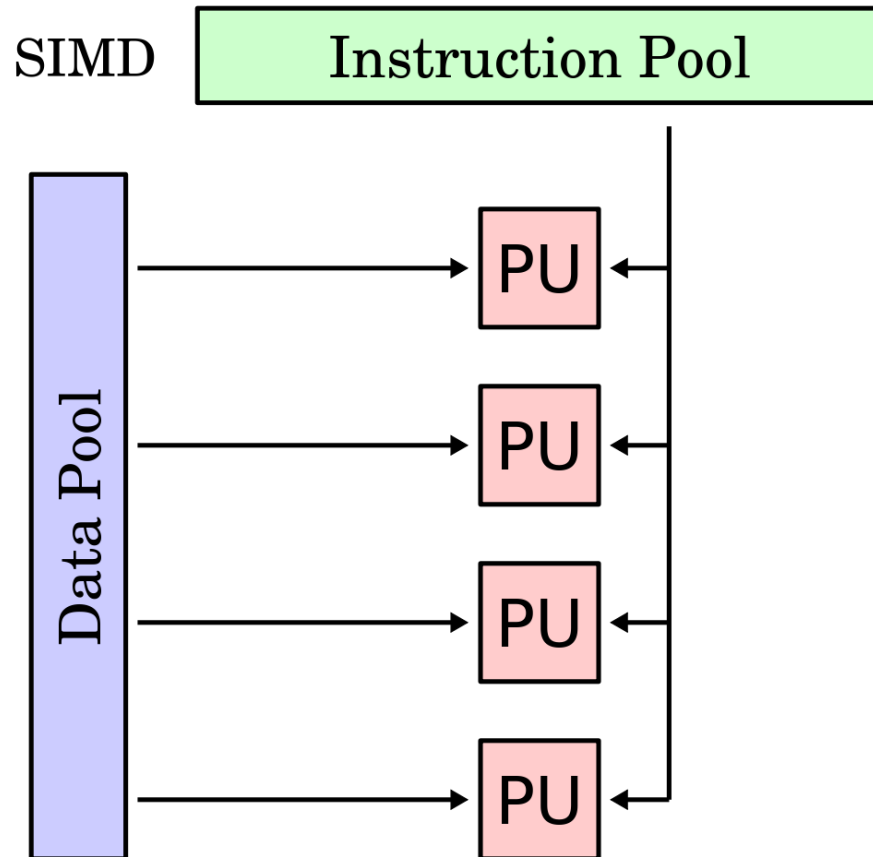
Parallel and Distributed Computers (Contd.)

SIMD Computers

- Single Instruction, Multiple Data (SIMD) is a type of parallel computing architecture that performs the same operation on multiple data elements simultaneously
- A Single-Instruction Multiple-Data (SIMD) computers are highly parallel machines, employing large arrays of simple processing elements
- In a SIMD machine, each processing element has a small amount of local memory
- The instructions executed by the SIMD computer are broadcast from a central instruction server to every processing element within the machine
- In this way, each processor executes the same instruction as all other processing elements within the machine
- Since each processor executes the instruction on its local data, all elements within the data structure are worked upon simultaneously
- The SIMD machine is generally used in conjunction with a conventional computer

Parallel and Distributed Computers (Contd.)

SIMD Computers





Parallel and Distributed Computers (Contd.)

SIMD Computers

- An example of this was the **Connection Machine (CM-1)** by Thinking Machines Corporation that used either a **VAX** minicomputer or a Silicon Graphics or Sun workstation as the "host" computer
- The **CM-1** was a finely grained SIMD computer with up to 64K of processing elements that appeared as a block of 64K of "intelligent memory" to the host system
- An application running on the host downloaded a **data set into the processor array** of the CM-1, each processor within the CM-1 acting as a single memory unit
- The host then issued **instructions to each processing element of the CM-1 simultaneously**. After the computations were completed, the host then read back the result from the CM-1 as though it were conventional memory



Parallel and Distributed Computers (Contd.)

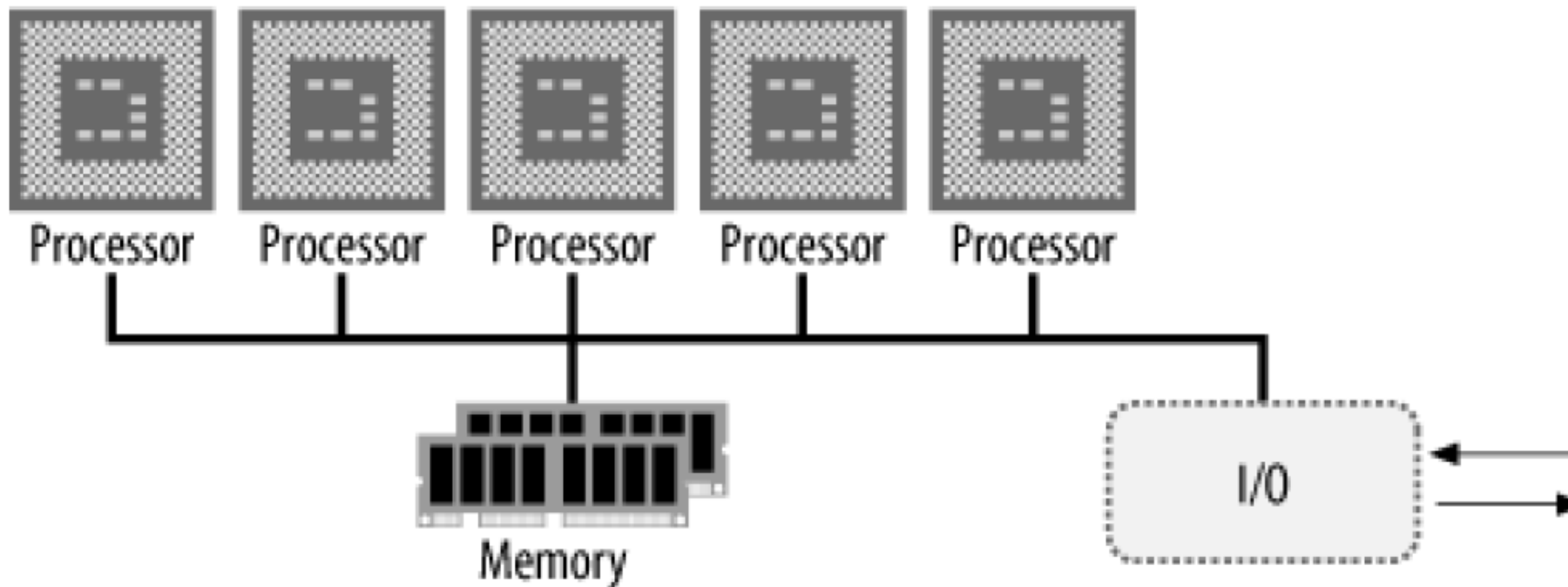
MIMD Computers

- Multiple Instruction, Multiple Data (MIMD) is a type of parallel computing architecture where multiple processors operate independently, each with its own control unit and memory
- The other major form of parallel machine is the Multiple-Instruction Multiple-Data (MIMD) computer
- These machines are typically coarsely grained collections of semi-autonomous processors, each with their own local memory and local programs
- An algorithm being executed on an MIMD computer is typically broken up into a series of smaller sub-problems, each executed on a processor of the MIMD machine
- By giving each processing element in the MIMD machine identical programs to execute, the MIMD machine may be treated as an SIMD computer
- MIMD computers tend to use a smaller number of very powerful processors, rather than many less powerful ones
- MIMD computers can be of one of two types: Shared-memory MIMD and Message-passing MIMD

Parallel and Distributed Computers (Contd.)

MIMD Computers

- Shared-memory MIMD systems have an array of high-speed processors, each with local memory or cache, and each with access to a large, global memory





Parallel and Distributed Computers (Contd.)

MIMD Computers

- The **global memory** contains the **data and programs** to be executed by the machine. Also in this memory is a **table of processes (or sub-programs) awaiting execution**
- Each **processor** will fetch a process and associated data into its local memory or cache and **will run semi-autonomously of the other processors in the system**
- The **process communication** also takes place through the global memory
- A speed advantage is gained by **sharing the program among several, powerful processors**
- However, **logic within the system must arbitrate between processors for access to the shared memory and associated shared buses of the system**
- In addition, allowances must be made for a processor attempting to access data in global memory that is out of date.



Parallel and Distributed Computers (Contd.)

MIMD Computers

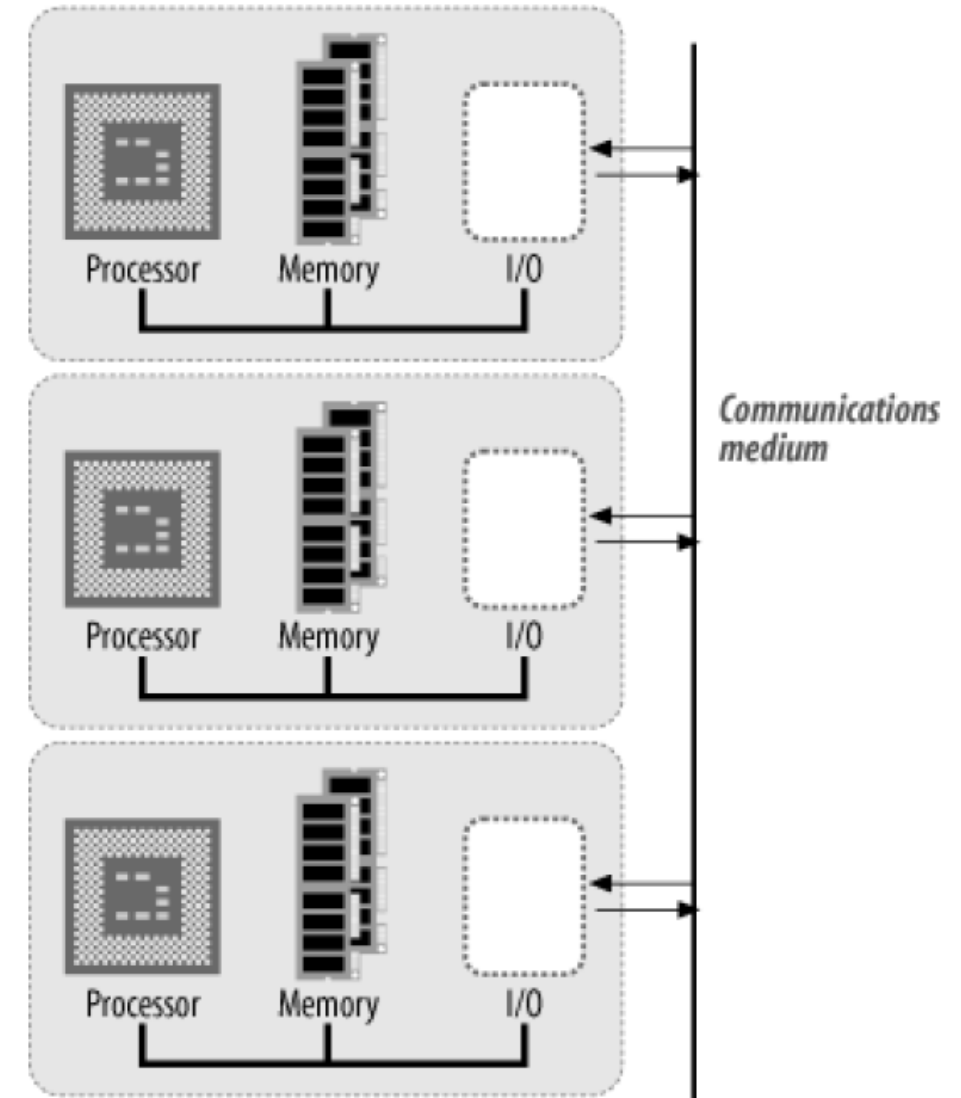
- If processor A reads a process and data structure into its local memory and subsequently modifies that data structure, processor B attempting to access the same data structure in main memory must be notified that a more recent version of the data structure exists. Such arbitration is implemented in processors like the (now extinct) Motorola MC88110, which was intended for use in shared-memory MIMD machines.

Parallel and Distributed Computers (Contd.)

MIMD Computers

Message-passing MIMD

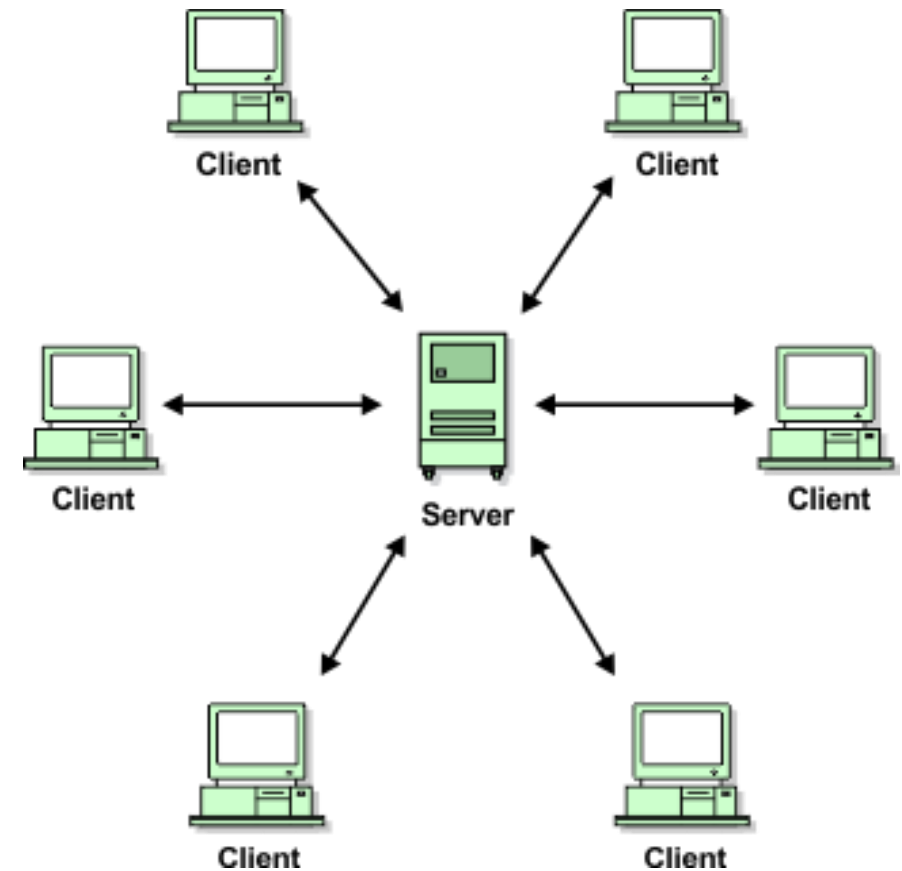
- In this system, each processor has its own local, main memory. No global memory exists for the machine
- The processors may communicate through a single, shared bus (such as Ethernet, CAN, or SCSI) or by using a more elaborate inter-processor connection architecture, such as 2-D arrays, N-dimensional hypercubes, rings, stars, trees, or fully interconnected systems
- MIMD machines do not suffer the bus-contention problems of shared-memory machines. However, the most effective and efficient means of interconnecting the processing nodes of a message-passing MIMD machine is still a major area of research



Parallel and Distributed Computers (Contd.)

Distributed Computers

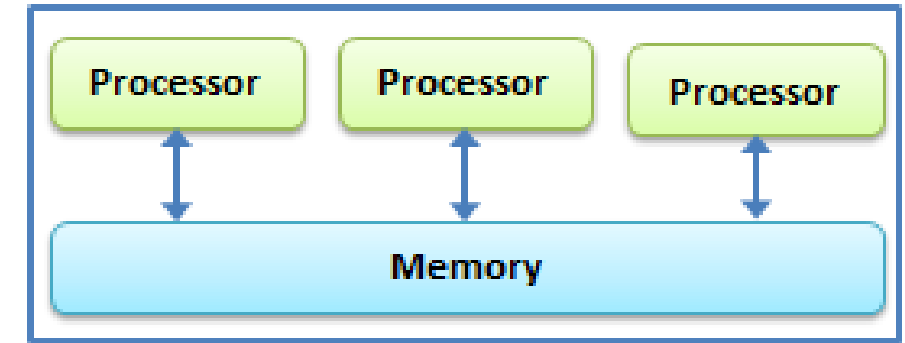
- A distributed machine is composed of individual computers networked together
- Distributed machines are common in the embedded world. A collection of small processing nodes may be distributed across a factory, providing local monitoring and control, and together forming a parallel machine executing the global control algorithm
- The avionics of commercial and military aircraft are also distributed parallel computers



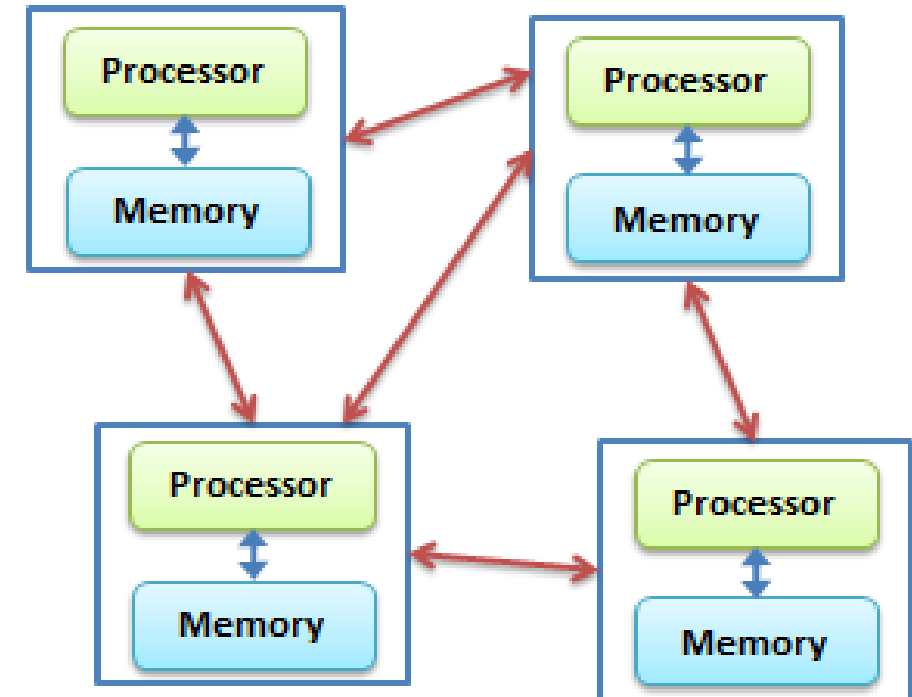
Parallel and Distributed Computers (Contd.)

Parallel VS. Distributed Computers

Parallel system	Distributed system
Single computer is used.	Multiple computers are used.
A shared memory or distributed memory can be used.	Only distributed memory can be used.
Multiple tasks are performed by multiple processors at same time.	Multiple tasks are performed by multiple computers at same time.
Communication between the processors is done through bus.	Communication between the computers is done through message passing.



Parallel Computing



Distributed Computing

*Thank
you*

